

LWK Sistemas — Estrutura do Projeto

Atualizado em 21/07/2026

Stack Tecnológica

Camada	Tecnologia	Versão
Backend	Django + DRF	6.0.7 + 3.17
Frontend	Next.js + React + TypeScript	16.2 + 19.2 + 6.0.3
Estilização	Tailwind CSS	4.x
Banco de dados	PostgreSQL (Railway)	16+
Cache	Redis (Railway)	7+
Deploy API/Workers	Railway	Docker
Deploy Frontend	Vercel	Node 20
Runtime	Python 3.12 / Node.js 20	LTS

Arquitetura

- **Multi-tenant:** cada loja possui schema PostgreSQL próprio (loja_<cpf_cnpj>)
- **Isolamento:** models com `LojalsolationMixin` + `LojalsolationManager`
- **Apps por tipo de loja:** `clinica_beleza`, `crm_vendas`, `cabeleireiro`, `hotel`, `restaurante`
- **Service layer:** lógica de negócio em `*_service.py`, não em `views/serializers`
- **API RESTful:** DRF com `APIView`, `GetObjectMixin`, paginação retrocompatível

Estrutura de Diretórios

- `backend/` – Django (API, models, services, migrations)
- `backend/clinica_beleza/` – App principal (consultas, recibos, retorno, agenda)
- `backend/crm_vendas/` – CRM de vendas (leads, propostas, contratos)
- `backend/asaas_integration/` – Integração financeira (cobranças, NFS-e)
- `backend/core/` – Configurações Django, middlewares, utilitários
- `frontend/` – Next.js 16 (App Router)
- `frontend/app/` – Páginas e rotas (App Router)
- `frontend/components/` – Componentes React por módulo
- `frontend/hooks/` – Custom hooks por domínio
- `frontend/lib/` – Utilitários, API clients, helpers
- `.github/workflows/` – CI/CD (testes, audit de segurança)
- `.railway/` – Configuração Railway (deploy backend)

Fluxo de Deploy

1. Verificar backend: `python3 manage.py check`
2. Verificar frontend: `getDiagnostics` nos arquivos alterados
3. Git: `git add` → `git commit` → `git push`
4. Backend: `npx railway up --detach` (raiz do projeto)

5. Frontend: `npx vercel --prod --yes` (raiz do projeto)
6. Se criou tabela: `SQL CREATE TABLE` em todos os schemas

Ambientes

Ambiente	URL	Branch
Produção	https://lwksistemas.com.br	main
Beta/Staging	https://beta.lwksistemas.com.br	staging
API Produção	https://api.lwksistemas.com.br	main

Módulos do Sistema

Clínica da Beleza (clinica_beleza)

- Agenda de consultas com bloqueios e horários por profissional
- Consultas: início, atendimento, evolução, finalização
- Recebimento: formas de pagamento múltiplas, desconto, retorno gratuito
- Recibos: PDF, email, WhatsApp (com info de retorno/prazo)
- Retorno gratuito: por consulta e por procedimento (prazo configurável)
- NFS-e: emissão via Asaas
- Protocolos de atendimento e procedimentos
- Fotos do paciente (QR code para upload)
- Documentos clínicos e prescrições (integração Memed)
- Produtos por consulta
- Histórico completo do paciente

CRM de Vendas (crm_vendas)

- Leads e oportunidades com pipeline visual
- Propostas e contratos com assinatura digital
- Financeiro: contas a receber/pagar, parcelas
- Campanhas de marketing (WhatsApp)
- Dashboard com métricas de conversão
- NFS-e integrada

Integrações

- Asaas: cobranças, boletos, PIX, cartão, NFS-e
- WhatsApp: envio de recibos, lembretes, campanhas
- Memed: prescrições médicas digitais
- Email: recibos, notificações

Convenções de Código

Backend

- Views: `APIView` + `GetObjectMixin` (elimina try/except repetido)
- Service layer: lógica de negócio em `*_service.py`
- Mapeamento: `map_field_names()` de `views_base.py`
- Resolução de loja: `resolve_loja_id_from_request()`
- Paginação: `paginate_queryset()` retrocompatível

- Validação: `python3 manage.py check` (0 issues)

Frontend

- Componentes: "use client" no topo de páginas interativas
- Formulários grandes: página dedicada (não modal)
- Modais: apenas para ações simples (confirmação, seleção)
- Cores: variáveis CSS temáticas por tipo de loja
- TypeScript strict mode habilitado
- Validação: `getDiagnostics` antes de commit

Convenções de Commit

- `feat(modulo)`: nova funcionalidade
- `fix(modulo)`: correção de bug
- `refactor(modulo)`: refatoração sem mudança de comportamento
- `perf(modulo)`: otimização de performance

CI/CD e Segurança

- GitHub Actions: testes backend (Django) + frontend (Vitest) + E2E (Playwright)
- Audit: `pip-audit` (backend) + `npm audit` (frontend) — bloqueia em HIGH
- Bandit: análise estática Python (bloqueia severidade alta)
- Dependabot: atualizações automáticas de segurança